

CW3 SE Report Group 22

Anonymous submission for the SE component of Coursework 3.

1 Task 1: Design Choices and Implementation Notes

1.1 Implemented Use Cases

This submission implements the group-22 main-system use cases:

- Log in
- Log out
- Register entertainment provider
- Create event
- Search for performances
- View performance
- Book performance
- Edit preferences
- Cancel performance
- Cancel booking

1.2 System Structure

Briefly describe the main architecture of the system. For example:

- the main packages and their responsibilities
- the role of controllers, models, view, and external/integration components
- how data is currently stored during execution

Write here:

1.3 Alignment with the Provided UML

State whether the implementation matches the provided class and sequence diagrams exactly. If there were any points where the diagrams could not be followed directly, explain:

- what the mismatch was
- why it could not be implemented exactly as drawn
- what design change was made in code
- how the UML could be updated to stay consistent with the implementation

Write here:

1.4 Key Design Choices

Summarise the most important choices you made when implementing the main system. For example:

- handling of input validation
- object ownership and shared state
- controller responsibilities
- interaction with mock external systems
- error handling strategy in the text interface

Write here:

1.5 Quality Considerations in the Implementation

Briefly mention implementation choices that support quality attributes such as security, privacy, or reliability. This can overlap with the later reflection, but here the focus should be on concrete system decisions.

Write here:

2 Task 5: Reflection

2.1 Reflection on Teamwork

The Experience Our team organised the implementation around GitHub pull requests rather than direct commits to the main branch. We created feature branches, opened pull requests for all changes, and used automated checks before review and merge. We also agreed early on the controller and model interfaces so that different use cases could be implemented in parallel with fewer clashes. This gave us a predictable workflow for coding, reviewing, and integrating work.

Reflection on Action This worked well because it reduced accidental breakage and made each change easy to inspect. The required checks on each pull request forced us to keep the project buildable: Maven tests, Java formatting, and LaTeX report generation all had to succeed before merge. Pre-commit automation also pushed formatting and generated report updates back to the pull-request branch, which reduced manual clean-up. Branch protection, required status checks, linear history, blocked force pushes, and squash-and-merge together kept the repository history much cleaner than in previous coursework. The main disadvantage was speed: even small changes took longer because they had to pass the same workflow, and debugging CI failures occasionally delayed progress.

Theory We learned that strong repository rules can improve teamwork, not just code quality. A pull-request-first workflow gave us both review discipline and a shared understanding of interfaces and integration points.

Preparation Next time we would keep the same protected-branch workflow, but add a lightweight board for tracking dependencies between use cases so that blocked work is visible earlier and review can be scheduled more evenly.

2.2 Reflection on the Quality of the Work and Quality Attributes

The Experience We think the strongest part of our work was the combination of implementation and testing rather than implementation alone. The main three-person-group use cases were built into a working text-based system, then reinforced with unit tests, use-case-specific system tests, menu-level tests, and longer end-to-end chains. We also spent time aligning the code with the supplied UML and later clarifications instead of treating the diagrams as disposable.

Reflection on Action The quality attribute we addressed most clearly was reliability. We tried to make invalid states and incorrect user actions visible and recoverable through explicit validation, controlled retry loops, and defensive handling of failure branches such as failed payment or refund flows. This was supported by tests for both normal paths and error paths, for example invalid identifiers, unavailable tickets, cancellation rules, and refund edge cases. We also improved reliability of the development process itself by requiring all checks to pass before merge. A second quality attribute was maintainability: controllers, models, view abstractions, utility parsers, CI packaging, and report tooling were all separated so that responsibilities stayed relatively clear.

Theory We learned that quality came less from any single design decision and more from repeatedly tightening mismatches between code, tests, and documentation.

Preparation With more time, we would measure coverage formally, expand Javadoc further,

and remove or refactor more residual placeholder APIs so the final codebase looked less like an extended coursework skeleton and more like a fully finished system.

2.3 Reflection on Tools, Agile Approaches and Practices

The Experience We chose GitHub as the centre of the workflow because it combined version control, pull requests, review, and CI in one place. We used Maven for building, testing, and packaging, pre-commit for formatting hooks, and GitHub Actions for automated checks. We also created a LaTeX build workflow so the report PDF was generated automatically instead of maintained manually.

Reflection on Action This toolchain matched an agile, incremental style of work. Small branches and pull requests allowed us to build use cases one at a time while keeping the system in a releasable state. Automated formatting reduced style arguments, while required checks made test failures visible immediately. The LaTeX workflow was useful because it turned the report into another reproducible build artifact rather than a separate manual task. The strongest practice here was continuous integration combined with branch protection: no direct commits to the protected branch, required checks before merge, squash merge for tidy history, linear history, and blocked force pushes. The downside was overhead. For coursework-scale changes, setting up and maintaining this pipeline took time, and a few CI issues required debugging infrastructure rather than application logic.

Theory We learned that these tools were most useful when combined as a system: pre-commit, CI, pull requests, and branch protection each reinforced the others.

Preparation Next time we would keep the same workflow, but add clearer dashboard-style tracking of open tasks and review ownership so that process information is as visible as build information.

2.4 Screenshots of Tools and Practices

The coursework document explicitly asks for screenshots in the tools/agile/practices reflection. Insert screenshots of the tools you actually used, such as issue tracking, Kanban boards, version control history, CI runs, or planning boards.

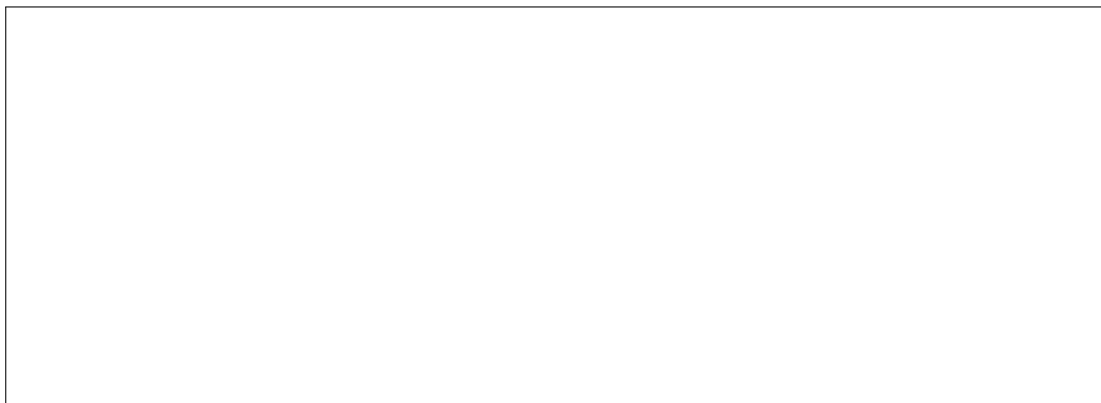


Figure 1: Screenshot placeholder 1. Replace with a real screenshot of a tool or practice used by the team.

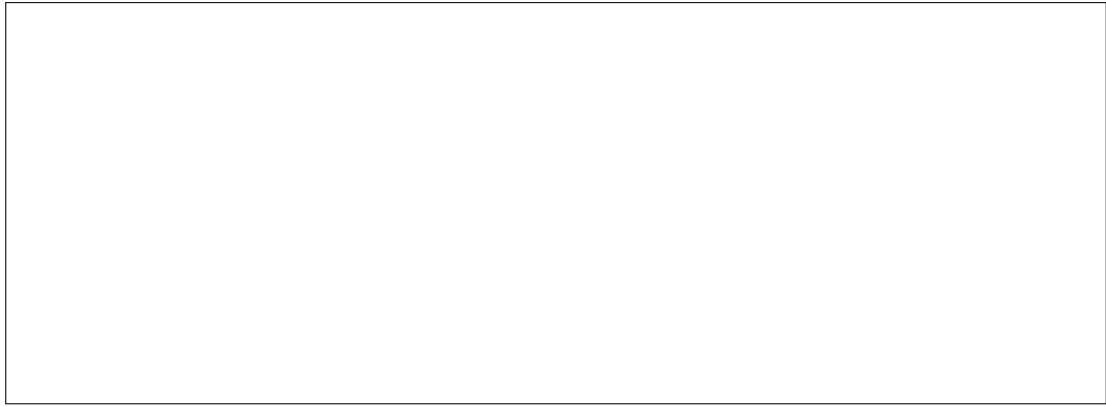


Figure 2: Screenshot placeholder 2. Replace with another real screenshot if needed.

3 Submission Checklist

Before exporting the PDF, check the following:

- The file is named `CW3SEReportGroup22.pdf`.
- The document is anonymous.
- The declaration of work is not included here.
- Task 1 choices are described.
- All three reflection parts are included.
- Each reflection part is at most 250 words.
- The tools/agile section includes screenshots.